

FlyNext: your most reliable travel companion!

FlyNext is a robust travel search platform tailored to simplify the process of booking flights and hotels. With an intuitive interface, FlyNext enables users to effortlessly search, compare flights across various airlines, find accommodations matching their preferences, and organize trips—all within a single platform. FlyNext leverages real-time data to provide accurate, up-to-date availability and pricing information, ensuring efficient and seamless travel planning. Users can refine searches using advanced filters such as price, departure time, duration, layovers, hotel amenities, ratings, and location. With a minimalistic design, FlyNext aims to enhance the travel search experience, offering both convenience and flexibility for a diverse range of travelers. Additionally, the platform integrates secure booking options, making it a one-stop destination for planning everything from short getaways to extended vacations.



About the project

This project is presented from the business owner's perspective, assuming limited technical familiarity with concepts like web development, Next.js, and APIs. Thus, the requirements are described non-technically, and your task is to design and implement a web application that fulfills them. There is much room for creativity, and you get decent freedom with respect to the details and various tools you can use for this project. Still, you **must** use Next.js, Prisma, REST, React, TailwindCSS, and Docker. Beside these, you can utilize any existing package, library, and open-source code to improve your app.

The project is divided into two parts. In part 1, you will use Next.js, Prisma, and REST Framework to implement the backend of the project. In part 2, you will implement the frontend using React and TailwindCSS and connect the backend to it. You will also use Docker to make your full application portable and deploy it on the internet. The eventual outcome of this project is a deployed, portable, and fully-functional web application that meets the user stories listed in an upcoming section.

Groups

This project is scoped for groups of three members. Groups of two would be a suitable option if some members have prior web development experience. You can also choose to work individually, which is strongly discouraged as the load will be overwhelming.

Note: The number of members will not affect our grading criteria. That is, your submission will be graded the same way, regardless of how many team members are there.

The IBS

All project-related activities—such as team registration, code submission, and interview scheduling—will be managed through [the IBS system](#). Begin by logging into the IBS system and registering your group. To register, invite your teammates using the system's group management feature. Once your group is registered, a GitLab repository will automatically be created for your team. Use this repository to push your code throughout the project. Monitor Piazza on detailed instructions for using IBS.

Deadline

The submission deadline for Part 1 is ~~Friday, March 7th, at 22:00~~ **Saturday, March 8th, at 22:00**. Late submissions are subject to a 10% penalty per day, up to a maximum of three days.

Students registered with Accessibility Services may qualify for deadline extensions. In that case, email the instructor at least one week before the deadline and request the extension. Note that individual entitlements will be adjusted to the size of the team. For example, if one member from a team of two is entitled to 7 days of extensions, the team will be granted $[7 \div 2] = 4$ days.

Academic Integrity

Honesty and fairness are fundamental to the University of Toronto's mission. Plagiarism is a form of academic fraud and is treated with utmost seriousness. You are expected to review the handout [How Not to Plagiarize](#) and to be familiar with the [Code of Behavior on Academic Matters](#).

You may use online resources, including open-source codes and generative AI tools like ChatGPT, to support your project. However, **sharing any portion of code with other teams, whether giving or receiving, is strictly prohibited.**

Note that all work that was not written by you or your teammates must be cited in the code, including open-source codes and code created by generative AI.

Mentor sessions

You can attend mentor sessions with a TA to discuss your ideas, ask questions, or seek help on any project-related issue. Note that the total number of sessions are limited and will be filled on a first-come-first-serve basis. Details on how to find a session will be posted on Piazza.

Interviews

Each part will be graded separately through interview sessions with TAs. Attending the interview is absolutely necessary for obtaining marks from of the project. All team members must be present at the meeting, and the **absent members will not get any marks** for the part. Instructions on booking interviews will be announced on Piazza.

Note that TAs will clone your code from the IBS and attempt to run it from scratch on their own machines. It is important that your submitted code is ready-to-go.

Participation

Each team member must do a fair amount of work on this project. We will assess each member's participation by investigating the commits they created and asking detailed questions about the code at the interviews. Even though it is a group project, your final grade is individual.

Submission

You should submit your work to your group's Gitlab repository on the IBS. Find out your repo's URL by logging into the IBS. Note that there is a separate repository for each part.

Correct use of git is part of your assessment. Commit your changes now and then, when a small logical change has been made. Do not leave your commits to the very last minutes before the deadline. Always write informative commit messages.

Environment

Your application must run in an Ubuntu 22.04 machine that has Node.js 20+ and SQLite3 installed. It is fine if you are developing your code on other operating systems, although Windows is strongly discouraged. However, it is your responsibility to make sure that your server and scripts (described later) run without any problem on Ubuntu 22.04 before submitting.

FlyNext features

The following sections outline the features to be included in FlyNext. These features are presented as **user stories** to reflect the experience from the perspective of imaginary users. Each story is written in non-technical terms, leaving specific design and implementation details to you.

User stories are categorized by user types:

As a visitor: Features available to all website visitors, logged in or not.

As a user: Features accessible only to authenticated users.

As a hotel owner: Features designed specifically for users managing hotels on the platform.

Important note: Some of the features described below are not applicable in this part of the project. However, it is essential to understand the full scope of the application to design it effectively.

Accounts

As a user, I want to sign up, log in, log out, and edit my profile. Profile information includes first and last name, email, profile picture, and phone number. Authentication should be handled with a proper JWT setup.

Flights search

As a visitor, I want to search for flights by specifying a source, destination, and date(s). Source and destination could be either a city or an airport. I want to search for one-way or round-trip flights.

As a visitor, I want to view flight details, including departure/arrival times, duration, and layovers.

As a visitor, I want to have an auto-complete dropdown to suggest cities and airports as I type in the source or destination field.

As a visitor, I want a date picker dropdown to easily select travel dates.

Hotel management

As a user, I want to add my hotel to the platform. A hotel has name, logo, address, location, star-rating, and several images.

As a hotel owner, I want to define room types, with each type having a name (e.g., twin, double, etc.), amenities, prices per night, and several images.

As a hotel owner, I want to view and filter my hotel's booking list by date and/or room type.

As a hotel owner, I want to cancel a hotel reservation, so that I have flexibility in managing my hotel.

As a hotel owner, I want to view room availability (per room type) for specific date ranges to better understand occupancy trends..

As a hotel owner, I want to update the number of available rooms of each type in my hotel. If availability decreases, it may require canceling some existing reservations.

Hotel search

As a visitor, I want to search for hotels by check-in date, check-out date, and city. I also want to filter them by name, star-rating, and price range. Search results should display in a list that shows the hotel information, starting price, and a location pinpoint on a map. The results should only reflect available rooms.

As a visitor, I want to view detailed hotel information, including room types, amenities, and pricing.

As a visitor, I want to view the availability and details of different room types for my selected dates in a selected hotel.

Booking

As a user, I want to book an *itinerary* that includes a flight (one-way or round-trip) and/or a hotel reservation.

As a user, I want to a **checkout page** that displays all details about my itinerary (flight and/or hotel), collects my credit card information, validates the card details, and finalizes the booking if everything is correct. Note that *Validation* means statically checking the validity of the card number and expiry date. The card should not (and cannot) be charged for real.

As a user, I want to see hotel suggestions for the city if I am flying to. I also want to see flight suggestions if I am about to book a hotel stay. Both suggestions must have a link to take me to the main hotel/flight search page with pre-filled inputs, while preserving my current, in progress order.

As a user, I want to receive a minimal, PDF invoice for my trip booking, so that I have a record of the booking and transaction.

As a user, I want to view my bookings, so that I can easily access my itinerary and booking information.

As a user, I want to cancel all or specific parts of a booking, giving me flexibility in managing my trips.

As a user, I want to **verify** my flight booking to ensure the flight schedule remains as planned.

Notifications

As a user, I want to receive notifications when I book a new itinerary, and when there are external changes to my booking (e.g., cancellation by me or hotel owner).

As a hotel owner, I want to receive notifications when a new booking is made for my hotel.

As a user or hotel owner, I want to see the number of unread notifications as a badge and have it updated as I read them.

User experience

As a visitor, I want a clean and intuitive user interface that allows me to navigate the platform effortlessly.

As a visitor, I want the website to be rendered well in different screen sizes (e.g., monitors, laptops, tablets, and mobile devices), ensuring accessibility and usability across devices.

As a visitor, I want to option to toggle between dark and light modes, so that I can choose the most comfortable viewing experience.

Advanced Flights System

Unlike hotels, flight data is not stored in FlyNext's internal database. Instead, FlyNext integrates with a centralized external system called Advanced Flights System (AFS) for managing flight-related operations. AFS provides APIs to retrieve data on cities, airports, airlines, and flight bookings. AFS is the repository of all flights, and you will search, book, and verify through it.

The system is hosted at <https://advanced-flights-system.replit.app>. The Postman collection of the APIs is available [here](#) [here](#). Moreover, Swagger-style API docs can be found [here](#) [here](#). They can be imported to [Swagger Editor](#). The list of APIs, information about authorization, API descriptions, and request and response formats are all outlined there.

AFS' source code is located [here](#). You do **not** need to run it locally. However, it is recommended to refer to the code as an example of a full-fledged Next.js application that uses Prisma and TypeScript.

Important note: The users must **never** directly interact with the AFS API. All communication with AFS must go through your backend server to protect third-party API keys and ensure secure integration. Lecture 8 contains an extensive discussion on the necessity of protecting third-party APIs. Thus, during the interview, the TAs will only call your backend server (not any other server). You will **not** receive any marks for APIs that are hosted elsewhere.

Authorization

All AFS APIs are authenticated. They require the `x-api-key` header, which must be equal your API key. Your API key is the output of running the `sha-256` hash on your student number. This is an obscure but unique number that can be obtained [here](#). Keep that number secure and do **not** expose it to others, including the client apps.

Note that you must put your API key into the header, exactly as produced by the hashing function -- without quotations, prefixes, or suffixes.

Part 1 deliverables

In this part, you will develop the Next.js backend for the project. You should design database models and implement API endpoints that fulfill the above user stories. All APIs must be written in the REST framework. You should also determine what user stories are relevant to the backend and only implement those.

This part's grading is completely separate and independent of the next part. You get to modify your backend code in for the next part. Your marks for part 1 will **not** update based on your submission in part 2.

Note 1: As potentially ambiguous as the handout gets, do not hesitate to ask your questions on Piazza and/or discuss them with TAs at mentor sessions.

Note 3: When initializing the Next.js project (using `npx create-next-app`), it is recommended to choose `yes` for the questions about the use of **App router**, **Typescript** and **TailwindCSS**. This will make sure that you will not need to manually re-configure the project at part 2. Ignore the `.ts` and config files and simply write your code in `.js` files. It will still work as expected.

Note 4: You need to implement the project using REST, Next.js, Prisma, React (next part), TailwindCSS (next part), and Docker (next part). Using alternative frameworks (such as GraphQL, Angular, Django, Sequelize, etc.) is not allowed.

Note 5: Even though submitting the `.env` file is discouraged in general, we ask you to push it to your repository to simplify setup for the TAs at the interviews.

Note 6: This part's marking scheme can be found [here](#).

What to submit

Push your entire Next.js project, as well as the following files (place them in the root folder of your repository):

A script named `startup.sh`: runs all preparations needed for your code to run in a new environment, including installing all required packages via `npm`, running all migrations, etc. Your startup script must also fetch the list of cities and airports (not flights) from the AFS and save them into your database. The list of cities is used for hotels as well, so it is better to host them locally.

A script named `run.sh`: starts your backend server.

A Postman collection (named `postman_collection.json`): a comprehensive and organized export of your APIs which is importable into Postman. The TAs will interact with your backend server via Postman. Be sure to include example POST data, query parameters, and headers for every request. Ensure the collection is self-explanatory and easy to navigate

Your collection should automatically set and use the tokens without the TAs having to manually paste them at every request.

The actual values of query parameters and POST data is not important; the TAs will change them at their discretion to test different scenarios.

An Swagger-compatible api docs (named `collection.openapi`): provides a nice visualization of all your APIs. The file should be importable to Swagger (<https://editor.swagger.io/>). It contains a full list of all API endpoints, where each endpoint includes a short description, allowed methods, payloads, and an example request and response. Note that several tools exist to convert Postman collections to OpenAPI and vice versa. Do not do it manually as it can take a huge amount of time. As an example, you can use [this tool](#).

A short document named `docs.pdf`: includes your model design with a brief explanation of database models, their fields, and the relationships. Having an ER or class diagram helps the illustration.

Refer to the AFS Postman and Swagger docs as an example.

Important note: Before submitting, ensure that your startup and run scripts function correctly in the described environment, and your collection can be imported into Postman without any issues. The TAs will test your application on clean instances of that environment. If you have developed on a different operating system, it is your responsibility to double-check that your server works on our environment as well.

Clarifications

Note 1: The user that creates a hotel becomes its owner (a.k.a manager). That is, any authenticated user can create hotels, and they will be the only one that can edit or delete it later.

Note 2: Using Docker in PP1 is allowed, as long as it's not hosting the code. In other words, if you want to use a specific database or software, you can use it via a Docker container.

Note 3: Currently, AFS does not have an API to support flight cancellation at the moment. Therefore, in this part, you do not need to support cancelling a flight booking in the itinerary. The API will be added in the next part, and the feature will be needed then.